

La taula *transaction* mostra major diversitat de format de les dades:

Table Name: transaction		Schema								
Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
id	VARCHAR(255)	☒	☒	☐	☐	☐	☐	☐	☐	
credit_card_id	VARCHAR(15)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
company_id	VARCHAR(20)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
user_id	INT	☐	☐	☐	☐	☐	☐	☐	☐	NULL
lat	FLOAT	☐	☐	☐	☐	☐	☐	☐	☐	NULL
longitude	FLOAT	☐	☐	☐	☐	☐	☐	☐	☐	NULL
timestamp	TIMESTAMP	☐	☐	☐	☐	☐	☐	☐	☐	NULL
amount	DECIMAL(10,2)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
declined	TINYINT(1)	☐	☐	☐	☐	☐	☐	☐	☐	NULL

En aquest cas cal assenyalar els camps *lat* i *longitude* que son amb FLOAT, donat el format de les coordenades geogràfiques, timestamp que és un camp de data/hora, amount és decimal, en aquest cas pot tenir 10 dígit i fins a 2 decimals i per últim declined que és TINYINT és un integer de màxim un byte de memòria. En aquest cas està definit com a un sol dígit. Segurament funciona com un booleà o per marcar algun estat definit internament per un valor.

Exercici 2

Utilitzant **JOIN** realitzaràs les següents consultes:

- Llistat dels països que estan generant vendes.

The screenshot shows a SQL IDE interface. The top part displays a query window with the following SQL code:

```

1 • USE transactions;
2
3 • SELECT country, SUM(amount)
4   FROM company
5  INNER JOIN transaction
6     ON company.id = transaction.company_id
7  GROUP BY country;
8
9
10

```

The bottom part shows the 'Result Grid' with the following data:

country	SUM(amount)
Germany	3466842.93
Australia	679418.68
United States	1069450.71
New Zealand	685567.09
Norway	785907.27
United Kingdom	3537120.34
Ireland	3530071.81

Below the result grid, the 'Output' window shows the execution log:

#	Time	Action	Message	Duration / Fetch
10	11:06:50	SELECT country, amount FROM company INNER JOIN transaction ON company.id = transaction.co...	100000 row(s) returned	0.000 sec / 0.422 sec
11	11:08:05	SELECT country, SUM(amount) FROM company INNER JOIN transaction ON company.id = transacti...	Error Code: 1140. In aggregated query without GROUP BY, expression #1 of SELECT list contains no...	0.016 sec
12	11:08:16	SELECT country, amount FROM company INNER JOIN transaction ON company.id = transaction.co...	100000 row(s) returned	0.000 sec / 0.359 sec
13	11:09:23	SELECT country, SUM(amount) FROM company INNER JOIN transaction ON company.id = transacti...	15 row(s) returned	0.407 sec / 0.000 sec

Hem fet un inner join de company i transaction a partir de les claus id i company_id. Agrupem els amounts per països fent un group by i ja tenim el llistat de països i les vendes.

- Des de quants països es generen les vendes.

```

10 • SELECT COUNT(DISTINCT country) AS Total_paisos
11 FROM company
12 INNER JOIN transaction
13 ON company.id = transaction.company_id;

```

Result 15 x

#	Time	Action	Message	Duration / Fetch
17	11:20:42	SELECT country FROM company INNER JOIN transaction ON company.id = transaction.company_id...	15 row(s) returned	0.109 sec / 0.000 sec
18	11:22:19	SELECT COUNT(DISTINCT country) AS Total_paisos FROM company INNER JOIN transaction ON ...	1 row(s) returned	0.093 sec / 0.000 sec

Per saber el número de països, fem una relació de noms únics per mitjà de *distinct* amb un *count* per a obtenir el total de països. L'*output* és una sola línia amb el número.

- Identifica la companyia amb la mitjana més gran de vendes.

```

16 -- companyia amb mitjana més gran de vendes
17 • SELECT company_name AS nom, AVG(amount) AS promig_transaccions
18 FROM company
19 INNER JOIN transaction
20 ON company.id = transaction.company_id
21 GROUP BY company_name
22 ORDER BY promig_transaccions DESC
23 LIMIT 1;

```

Result 19 x

#	Time	Action	Message	Duration / Fetch
22	11:28:32	SELECT company_name, AVG(amount) AS promig_transaccions FROM company INNER JOIN trans...	1 row(s) returned	0.485 sec / 0.000 sec
23	11:28:44	SELECT company_name AS nom, AVG(amount) AS promig_transaccions FROM company INNER JO...	1 row(s) returned	0.437 sec / 0.000 sec

Seguim amb el join de company i transaction i fem la mitja d'amount agrupades per empreses, les ordenem en ordre descendent i limitem la sortida a 1 empresa. Aquesta és la que té el màxim de mitjana de vendes.

Exercici 3

Utilitzant només subconsultes (sense utilitzar **JOIN**):

- Mostra totes les transaccions realitzades per empreses d'Alemanya.

```

27 -- ***** APARTAT 2 *****
28 -- transaccions realitzades per empreses d'Alemanya
29 • SELECT company_id, id
30 FROM transaction AS t
31 WHERE company_id IN -- és dins del conjunt de companyies alemanyes
32     (SELECT id FROM company
33      WHERE country = 'Germany');
34

```

company_id	id
b-2222	0F35A6C3-EDB0-41BE-808F-407F91FC1EA3
b-2222	0F4AF8B8-3E70-46F4-8BA9-DDF1D810ACA1
b-2222	0F646E8C-1A89-4505-92CC-A7FC82A401FF
b-2222	0FA47D77-7772-40D3-951D-9EBD5BA2AA80
b-2222	0FC032DD-01E5-4C20-88C0-650F2F52FE4E
b-2222	1001CA60-1259-4448-8AAB-3925D55C303E
b-2222	101EB281-4C45-4F7A-80DA-20567347E621
h-2222	10781436-70BF-476F-A219-4A8APF48F0A6

#	Time	Action	Message	Duration / Fetch
64	12:40:28	SELECT COUNT(company_id) FROM transaction AS t WHERE company_id IN -- és dins del conjunt ...	1 row(s) returned	0.015 sec / 0.000 sec
65	12:42:12	SELECT company_id, id FROM transaction AS t WHERE company_id IN -- és dins del conjunt de co ...	13291 row(s) returned	0.000 sec / 0.015 sec

Fem una consulta sobre transaction en la que (where fa de filtre) el país sigui Alemanya. En el where afegim la subconsulta definint el país tal com demana l'enunciat.

L'output és la llista de transaccions. Hi he afegit la id de la companyia. Si volguessim el nom, caldria una altra subconsulta si no podem fer INNER JOIN.

- Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.

```

35 -- Empreses amb transaccions per un amount superior a la mitjana de totes les transaccions.
36 • SELECT id, company_name -- para ver el amout habria que hacer una consulta correlacionada
37 FROM company
38 WHERE id IN
39     (SELECT company_id
40      FROM transaction
41      WHERE amount >
42          (SELECT AVG(amount)
43           FROM transaction)
44      );

```

id	company_name
b-2222	Ac Fermentum Incorporated
b-2226	Magna A Neque Industries
b-2230	Fusce Corp.
b-2234	Convallis In Incorporated
b-2238	Ante Iaculis Nec Foundation
b-2242	Donec Ltd
b-2246	Sed Nunc Ltd
h-2250	Amet Nibla Donec Corporation

#	Time	Action	Message	Duration / Fetch
62	12:36:59	SELECT id, company_name, amount FROM company WHERE id IN (SELECT company_id FROM...	Error Code: 1054. Unknown column 'amount' in 'field list'	0.000 sec
63	12:39:41	SELECT id, company_name -- para ver el amout habria que hacer una consulta correlacionada FRO...	100 row(s) returned	0.125 sec / 0.000 sec

Fem la consulta sobre company per que busquem el seu nom, però hem de filtrar per amount. Per tant en el where hi afegim la subconsulta a la taula transaction. Les dues van enllaçades pel camp id i company_id, que son les PRIMARY KEYS.

Per cercar les que estan per sobre de la mitjana, definim el filtre al segon where fent una segona subconsulta per trobar la mitjana d'amount (AVG(amount)).

- Llistat d'empreses que no tenen transaccions registrades,

```
46 -- llistat empreses que no tenen transaccions registrades
47 • SELECT company_name AS nom_empresa
48 FROM company
49 WHERE id NOT IN
50     (SELECT DISTINCT company_id
51      FROM transaction);
52 -- busco entre todas las empresas con transacciones cuáles no estan en la tabla company
53 -- el resultado es que todas tienen registros.
54
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

nom_empresa	
-------------	--

company 76 x

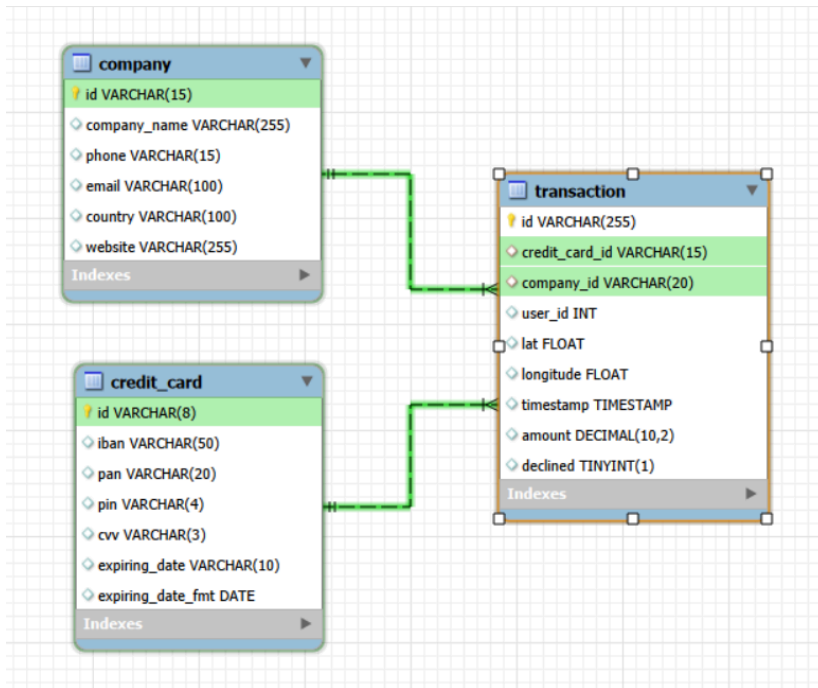
Output: To edit table data, the SELECT statement must include the primary key column(s)

#	Time	Action	Message	Duration / Fetch
95	13:04:57	SELECT company_name FROM company WHERE id NOT IN (SELECT DISTINCT company_id FR...	0 row(s) returned	0.062 sec / 0.000 sec
96	13:06:33	SELECT company_name AS nom_empresa FROM company WHERE id NOT IN (SELECT DISTINC...	0 row(s) returned	0.062 sec / 0.000 sec

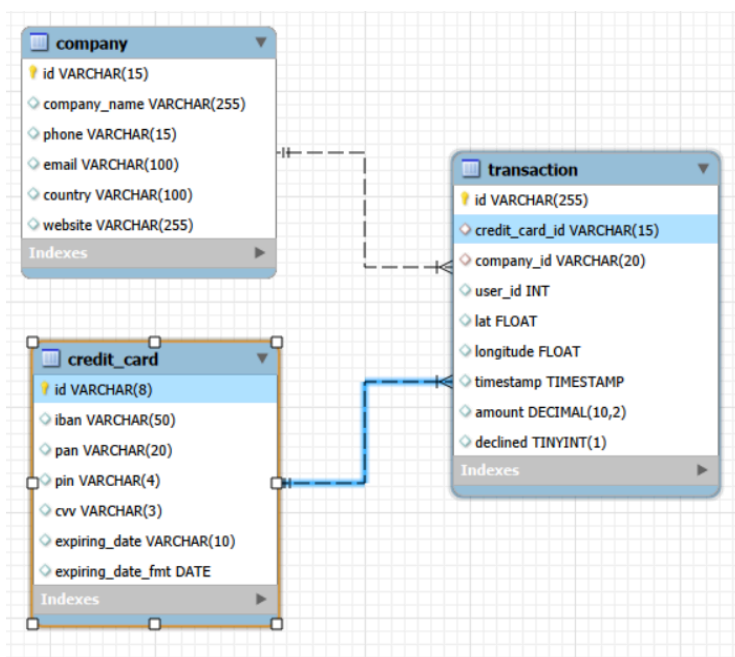
Busco les empreses de la taula *company* que no estan (no tenen cap registre) a transaction. La subconsulta llista totes les empreses de *company* i busco si hi ha alguna d'aquestes que NO estigui a transaccions. Em surt el camp buit. Això vol dir que totes tenen registres de transaccions.

Exercici 4

Dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada tarjeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.



Així és com queda la relació de la tres taules. `credit_card` es relaciona amb `transaction` 1 és a molts (la pota de gall és a `transaction`). La *primary_key* de `credit_card` és la `id`, que es relaciona amb `transaction` a partir de `credit_card_id`:



Ara `transaction` té com a *primary_key* `id`, com ja tenia i dues *foreign_keys*, tal com es mostra a continuació:

transaction - Table	
Table Name: transaction	
Foreign Key Name	Referenced Table
fk_transaction_creditcard	`transactions`.`credit_card`
transaction_ibfk_1	`transactions`.`company`

Per a la manipulació de les dades he tingut problemes a la hora d'omplir la taula credit_card per incompatibilitat del format DATE.

Seguint alguna ajuda externa he hagut de fer un:

```
SET expiring_date_fmt = STR_TO_DATE(expiring_date, '%m/%d/%y');
```

SET vol dir que escriu o defineix la columna expiring_date_fmt amb el format de data mes/dia/any. Si no, mySQL no em deixa avançar.

Exercici 5

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.

```

95 • SET SQL_SAFE_UPDATES = 0;
96 • update credit_card set id = 'CcU-2938' where iban = 'TR323456312213576817699999';
97 • SET SQL_SAFE_UPDATES = 1;
98
99 • select *
100 from credit_card
101 where id = 'CcU-2938';
102

```

id	iban	pan	pin	cvv	expiring_date	expiring_date_fmt
CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	10/30/22	NULL

credit_card 3

Output

#	Time	Action	Message	Duration / Fetch
6	10:43:35	update credit_card set id = 'CcU-2938' where iban = 'TR323456312213576817699999'	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0	0.000 sec
7	10:43:39	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec
8	10:45:53	select * from credit_card where id = 'CcU-2938'	1 row(s) returned	0.000 sec / 0.000 sec

Selecciono el id 'CcU-2938' i modifico l'IBAN pel de l'enunciat.

La modificació es fa per mitja d'un **UPDATE** taula **SET** id = 'iban'.

Llenço la consulta al final para mostrar que s'ha fet el canvi.

EXERCICI 6

En la taula "transaction" ingressa una nova transacció:

```

101 • show columns from transaction;
102 • SET FOREIGN_KEY_CHECKS = 0;
103 • INSERT INTO transaction(id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
104   VALUES ('10881D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', 999, 829.999, -117.999, 111.11, 0);
105 • SET FOREIGN_KEY_CHECKS = 1;
106 • select *
107   from transaction
108  where credit_card_id = 'CcU-9999';
109

```

The screenshot shows a database interface with a SQL editor at the top and a results pane below. The SQL editor contains the following queries:

```

show columns from transaction;
SET FOREIGN_KEY_CHECKS = 0;
INSERT INTO transaction(id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
VALUES ('10881D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', 999, 829.999, -117.999, 111.11, 0);
SET FOREIGN_KEY_CHECKS = 1;
select *
from transaction
where credit_card_id = 'CcU-9999';

```

The results pane shows the output of the last query, displaying a single row in the 'transaction' table:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
10881D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	999	829.999	-117.999	111.11	0	

Below the table, the 'Action Output' pane shows the execution details of the queries:

#	Time	Action	Message	Duration / Fetch
21	11:08:16	select * from credit_card where id = 'CcU-9999'	0 row(s) returned	0.000 sec / 0.000 sec
22	11:08:30	select * from transaction where id = 'CcU-9999'	0 row(s) returned	0.000 sec / 0.000 sec
23	11:08:58	select * from transaction where credit_card_id = 'CcU-9999'	1 row(s) returned	0.000 sec / 0.000 sec

Per afegir un registre a una taula he fet servir:

INSERT INTO taula(lista de columnes) **VALUES** (valors seguint ordre de la llista de columnes)

Després de manipular les dades a excel per poder escriure la consulta a Workbench, he fet la comprovació de que s'ha escrit el nou registre a *transaction*.

He hagut de deshabilitar la comprovació de les *foreign keys* per poder escriure el valor a la taula. Després d'introduir el valor les he tornat a habilitar.

A tenir en compte que no hi ha *timestamp* en aquest registre.

A la imatge es mostra el registre nou creat per mitja d'una consulta a través de la nova card_id introduïda.

EXERCICI 7

Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recorda mostrar el canvi realitzat.

```

110  -- **** APARTAT 7 ****
111
112  • ALTER TABLE credit_card DROP COLUMN Pan;
113  • SHOW COLUMNS FROM credit_card;
114

```

Field	Type	Null	Key	Default	Extra
id	varchar(8)	NO	PRI		
iban	varchar(50)	YES			
pin	varchar(4)	YES			
cvv	varchar(3)	YES			
expiring_date	varchar(10)	YES			
expiring_date_fmt	date	YES			

#	Time	Action	Message	Duration / Fetch
24	11:19:30	SHOW COLUMNS FROM credit_card	7 row(s) returned	0.000 sec / 0.000 sec
25	11:19:44	ALTER TABLE credit_card DROP COLUMN Pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.046 sec
26	11:19:54	SHOW COLUMNS FROM credit_card	6 row(s) returned	0.000 sec / 0.000 sec

Per eliminar una columna he fet servir **ALTER TABLE** taula **DROP** columna

La sortida de la consulta es pot veure que no hi ha la columna Pan, tal com demana l'enunciat.

EXERCICI 8

Dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules.

Abans de tot hem creat la base de dades i l'he seleccionat per al seu ús:

CREATE DATABASE nova_transaction; # aquí creo la base de dades
USE nova_transaction; # per fer servir aquesta base de dades d'ara endavant.

A continuació hi ha l'output de la creació de la taula transactions:

```

119
120  • CREATE TABLE transactions (
121      Id VARCHAR(40) PRIMARY KEY,
122      card_id VARCHAR(15),
123      business_id VARCHAR(6),
124      timestamp DATETIME,
125      amount DECIMAL(10,2),
126      declined TINYINT(1),
127      product_ids VARCHAR(50),
128      user_id INT,
129      lat DOUBLE,
130      longitude DOUBLE, -- usamos DOUBLE pq son coordenadas y necesitamos alta precisión
131      discount amount DECIMAL(10,2);

```

Field	Type	Null	Key	Default	Extra
Id	varchar(40)	NO	PRI		
card_id	varchar(15)	YES			
business_id	varchar(10)	YES			
timestamp_transaction	datetime	YES			
amount	decimal(10,2)	YES			
declined	tinyint(1)	YES			
product_ids	varchar(50)	YES			
user_id	int	YES			
discount	decimal(10,2)	YES			

#	Time	Action	Message	Duration / Fetch
39	12:05:09	ALTER TABLE transactions CHANGE timestamp_transaction DATETIME	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec
40	12:33:28	ALTER TABLE transactions CHANGE business_id VARCHAR(10)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec
41	12:33:41	SHOW COLUMNS FROM transactions	19 row(s) returned	0.000 sec / 0.000 sec

Primer he obert el CSV en excel per veure com son les dades. He copiat la primera fila que son els camps i a partir d'aquí he construït la taula definint cada camp segons tipus de dada i la seva longitud o mida.

La resta de taules les he creat de la mateixa manera, manualment amb codi.

Problemes trobats:

Les taules usuaris Europa i usuaris USA s'han hagut de fusionar primer fent un **UNION ALL**, ja que transaction tenia usuaris amb codis USA i codis Europa:

```

276
277 • CREATE TABLE dim_users AS -- fusiono las tablas de usuarios, dado que en transactions hay datos de USA y de Europa. es una nueva dimension de la 8800.
278 SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band
279 FROM users1
280 UNION ALL
281 SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band
282 FROM users2;

```

#	Time	Action	Message	Duration / Fetch
72	10:25:22	describe users_junts	13 row(s) returned	0.000 sec / 0.000 sec
73	10:28:52	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_users_junts FOREIGN KEY (user_id) ...	100001 row(s) affected Records: 100001 Duplicates: 0 Warnings: 0	3.110 sec
74	10:37:40	CREATE TABLE dim_users AS --fusiono las tablas de usuarios, dado que en transactions hay datos ...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.156 sec

El resultat és una taula amb 5000 usuaris.

Creo a continuació la *primary key* sobre la nova taula fusionada i la *Foreign_key* sobre *transaction* amb *ALTER TABLE*.

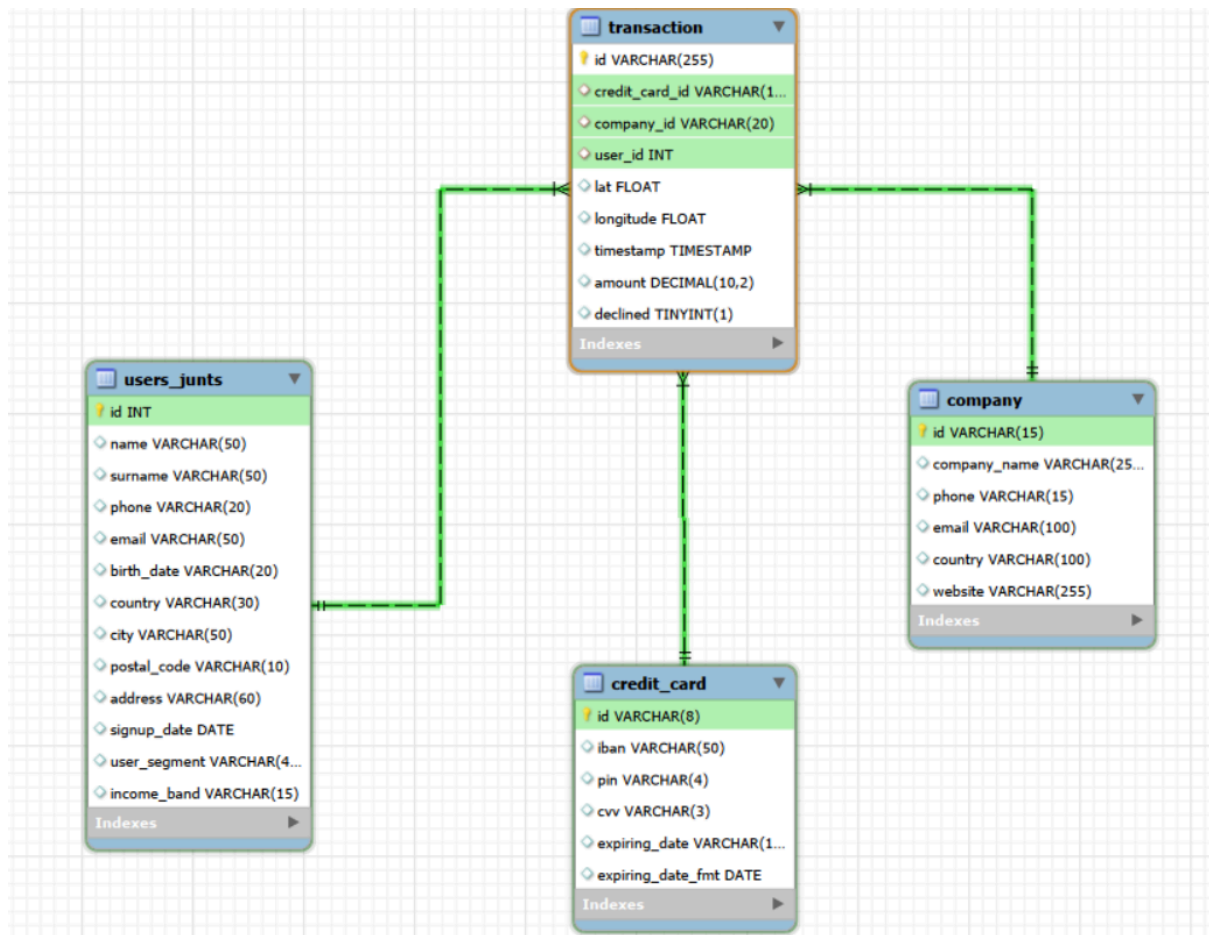
```

290 -- creo primary key
291 • ALTER TABLE users_junts ADD PRIMARY KEY (id);
292 • describe users_junts;
293
294 -- relaciono amb transaction
295 • ALTER TABLE transaction
296 ADD CONSTRAINT fk_transaction_users_junts
297 FOREIGN KEY (user_id) REFERENCES users_junts(id);
298
299
300

```

#	Time	Action	Message	Duration / Fetch
70	10:23:01	describe users_junts	13 row(s) returned	0.016 sec / 0.000 sec
71	10:25:10	ALTER TABLE users_junts ADD PRIMARY KEY (id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.172 sec
72	10:25:22	describe users_junts	13 row(s) returned	0.000 sec / 0.000 sec
73	10:28:52	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_users_junts FOREIGN KEY (user_id) ...	100001 row(s) affected Records: 100001 Duplicates: 0 Warnings: 0	3.110 sec

La relació de les taules en la base de dades *transactions* queda així:



Amb les corresponents *FOREIGN KEYS* a *transactions* per crear la relació de les taules:

transaction - Table		
Table Name: transaction		
Foreign Key Name	Referenced Table	Column
fk_transaction_creditcard	`transactions`.`credit_card`	
fk_transaction_users_junts	`transactions`.`users_junts`	
transaction_ibfk_1	`transactions`.`company`	

Exercici 9

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

The screenshot shows a SQL IDE with a query editor at the top and a results pane at the bottom. The query is as follows:

```
303 • SELECT id, name, surname
304 FROM users_junts
305 WHERE id IN (
306     SELECT user_id
307     FROM transaction
308     GROUP BY user_id
309     HAVING COUNT(user_id) > 80
310 )
311
312
```

The results pane shows a table with 4 rows and 3 columns: id, name, and surname.

id	name	surname
185	Molly	Gillam
289	Dxxwgi	Hwcru
318	Bnyr	Astuw
454	Sfzoh	Xgvfridxs

The bottom pane shows the 'Action Output' with the following log entries:

#	Time	Action	Message	Duration / Fetch
93	11:11:40	SELECT user_id FROM transaction GROUP BY user_id HAVING COUNT(user_id) > 80	4 row(s) returned	0.031 sec / 0.000 sec
94	11:11:49	SELECT user_id FROM transaction GROUP BY user_id HAVING COUNT(*) > 80	4 row(s) returned	0.016 sec / 0.000 sec
95	11:12:23	SELECT id, name, surname FROM users_junts WHERE id IN (SELECT user_id FROM transaction ...	4 row(s) returned	0.031 sec / 0.000 sec

Faig servir la taula de usuaris consolidats (*users_junts*) i la de *transactions*. Busco id i nom dels usuaris i en el filtre del *where* hi poso la subconsulta que apunta a *transaction* enllaçada amb les id's. Aquesta subconsulta és per trobar les id's que es repeteixen més de 80 vegades a les transaccions. Vol dir que ha fet més de 80 operacions. A la sortida tenim 4 usuaris.

Exercici 10

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

Faig un INNER JOIN entre *transaction* i *credit_card* primer i al filtre del nom de les companyies hi adjunto una subconsulta per filtrar en les empreses, la de Donec.

```

314 • SELECT credit_card.iban, AVG(amount) AS mitjana_amount
315 FROM transaction
316 INNER JOIN credit_card ON credit_card.id = transaction.credit_card_id
317 WHERE transaction.company_id = (SELECT id
318 FROM company
319 WHERE company_name = 'Donec Ltd')
320 GROUP BY credit_card.iban;
321
322

```

iban	mitjana_amount
XX911406401125586307586805	356.246667
SK9446370242474562577506	142.960000
XX776752917845952975555640	257.370000
XX413827362289719304908990	139.590000
XX347787246070769610780308	240.410000
XX688768436543090894854602	188.580000
MK28368851538688349	439.390000
PL76249283566852676343404576	541.560000
YV999999999999999999999999	180.210000

Result 57 x

Output

#	Time	Action	Message	Duration / Fetch
125	11:42:12	SELECT credit_card_id, AVG(amount) AS mitjana_amount FROM transaction INNER JOIN credit_card...	Error Code: 1055. Expression #1 of SELECT list is not in GROUP BY clause and contains nonaggrega...	0.000 sec
126	11:42:39	SELECT credit_card.iban, AVG(amount) AS mitjana_amount FROM transaction INNER JOIN credit_c...	371 row(s) returned	0.016 sec / 0.000 sec

Nivell 2

Exercici 1

Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.

```

322 -- **** Exercici 1 ****
323 • SELECT * FROM transaction;
324
325 • SELECT DATE(timestamp), SUM(amount) as suma
326 FROM transaction
327 GROUP BY DATE(timestamp)
328 ORDER BY suma DESC
329 LIMIT 5;

```

DATE(timestamp)	suma
2022-12-13	14337.44
2019-11-18	13591.32
2023-02-20	13332.59
2017-12-20	13318.43
2019-03-18	12680.95

Result 65 x

Output

#	Time	Action	Message	Duration / Fetch
135	12:01:11	SELECT DATE(timestamp), SUM(amount) as suma FROM transaction ORDER BY suma DESC LIM...	Error Code: 1140. In aggregated query without GROUP BY, expression #1 of SELECT list contains no...	0.000 sec
136	12:03:07	SELECT DATE(timestamp), SUM(amount) as suma FROM transaction GROUP BY DATE(timestamp) ...	5 row(s) returned	0.141 sec / 0.000 sec

Em lia el fet que em demana la data de cada transacció.

EL problema aquí és el format de la data. He fet servir DATE a la columna timestamp per poder treure la data en el format any-mes-dia.

A partir d'aquí he fet la consulta normal agrupant les *amount* per data i limitant a 5 entrades, els 5 dies amb més ingressos.

Exercici 2

Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.

```

347 • SELECT company_name, phone, country, DATE(transaction.timestamp) AS fecha, amount
348 FROM company
349 INNER JOIN transaction ON company.id = transaction.company_id
350 WHERE amount > 350 AND amount < 400 AND (
351     DATE(transaction.timestamp) = '2015-04-29'
352     OR DATE(transaction.timestamp) = '2018-07-20'
353     OR DATE(transaction.timestamp) = '2024-03-13')
354 ORDER BY amount DESC;
355
356
357

```

company_name	phone	country	fecha	amount
Aliquam PC	01 45 73 52 16	Germany	2024-03-13	399.84
Auctor Mauris Vel LLP	08 09 28 74 14	United States	2018-07-20	399.51
At Pede Corp.	06 14 48 33 15	Italy	2015-04-29	390.69
Aliquam PC	01 45 73 52 16	Germany	2024-03-13	388.29
Orci Adipiscing Limited	03 18 00 77 81	United Kingdom	2018-07-20	373.71
Fringilla LLC	08 29 15 93 57	New Zealand	2015-04-29	367.62
Pede Cum Ltd	07 62 26 48 38	Norway	2018-07-20	356.87
Auctor Mauris Vel LLP	08 09 28 74 14	United States	2024-03-13	353.75

Result 79 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
161	13:25:49	SELECT company_name, phone, country, DATE(transaction.timestamp) AS fecha, amount FROM c...	8 row(s) returned	0.062 sec / 0.000 sec
162	13:26:13	SELECT company_name, phone, country, DATE(transaction.timestamp) AS fecha, amount FROM c...	8 row(s) returned	0.109 sec / 0.000 sec

Faig un INNER JOIN per poder extreure la informació de les companyies i poder-ho ajuntar amb les transactions.

Exercici 3

Informació sobre la quantitat de transaccions que realitzen les empreses. Cal especificar si tenen igual o més de 400 transaccions o menys.

```

358 • SELECT count(transaction.id), company_name
359 FROM transaction
360 INNER JOIN company ON transaction.company_id = company.id
361 GROUP BY transaction.company_id, company.company_name;
362

```

count(transaction.id)	company_name
2401	Ac Fermentum Incorporated
410	Magna A Neque Industries
447	Fusce Corp.
1514	Convalis In Incorporated
472	Ante Iaculis Nec Foundation
449	Donec Ltd
1541	Sed Nunc Ltd
1511	Amet Nulla Donec Corporation
474	Nunc Nunc Dignissim Inc

Result 88 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
181	13:46:45	SELECT count(id), company_name FROM transaction INNER JOIN company ON transaction.compan...	Error Code: 1052. Column 'id' in field list is ambiguous	0.000 sec
182	13:48:11	SELECT count(transaction.id), company_name FROM transaction INNER JOIN company ON transact...	100 row(s) returned	0.234 sec / 0.000 sec

Exercici 4

Borrar registro de la tabla ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'

```

362
363 -- **** Exercici 4 ****
364
365 # ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD
366
367
368 • DELETE FROM transaction WHERE ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';

```

Output

#	Time	Action	Message	Duration / Fetch
187	13:54:35	SELECT ID FROM transaction WHERE ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	0 row(s) returned	0.000 sec / 0.000 sec
188	13:55:48	DELETE FROM transaction WHERE ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	0 row(s) affected	0.000 sec

Exercici 5

Crea una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.

```

371 • CREATE VIEW VistaMarketing AS
372 SELECT company_name, phone, country, AVG(amount) AS mitjana
373 FROM company
374 INNER JOIN transaction ON transaction.company_id = company.id
375 GROUP BY company_name, phone, country
376 ORDER BY mitjana DESC;
377
378 • SELECT * FROM VistaMarketing;
379

```

Result Grid

company_name	phone	country	mitjana
Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.867160
Pretium Neque Corp.	07 77 48 55 28	Australia	276.158330
Urnna Convalis Associates	06 01 24 77 04	United States	274.235011
At Associates	09 56 61 10 65	New Zealand	272.214870
Metus Vitae Associates	08 25 44 40 66	Australia	270.080965
Aliquet Diam Limited	02 76 61 47 46	United States	269.599181
Nec Luctus LLC	02 14 71 75 73	Norway	268.604837
Neque Tellus Incorporated	04 43 18 34 19	Ireland	267.850372
Tortor Minc Commodo Company	05 35 07 77 16	United States	267.836085

Output

#	Time	Action	Message	Duration / Fetch
194	14:04:15	CREATE VIEW VistaMarketing AS SELECT company_name, phone, country, AVG(amount) AS mitjan...	0 row(s) affected	0.015 sec
195	14:04:29	SELECT * FROM VistaMarketing	100 row(s) returned	0.547 sec / 0.000 sec

Hem creat primer la vista com una consulta normal amb un inner join. Després invoco la vista amb un select normal i la taula és la vista.

Nivell 3

Exercici 1

Estat de les targetes. Quines estan actives?

Si les tres últimes transaccions han estat declinades aleshores és inactiu.

Creem una taula per recollir l'estat de les targetes:

```

499 • CREATE TABLE credit_card_status AS
500 -- Una CTE (Common Table Expression) es una subconsulta con nombre,
501 -- definida con WITH, que se usa como si fuera una tabla en el SELECT
502 -- principal.
503 WITH ultimas AS (
504     SELECT
505         card_id,
506         declined,      -- 0 = aceptada, 1 = declinada
507
508         -- ROW_NUMBER() es una función de ventana: numera las filas
509         ROW_NUMBER() OVER (
510             -- PARTITION BY reinicia la numeración cada vez que cambia
511             PARTITION BY card_id
512             -- ORDER BY fecha DESC decide el criterio de numeración DENTRO de cada partición: la transacción más reciente
513             ORDER BY DATE(timestamp) DESC
514         ) AS orden      -- alias: posición de la transacción dentro de su propia tarjeta
515     FROM transaction
516 )
517 -- Fin de la CTE.
518
519 -- consulta principal, que sí usa la CTE.
520 SELECT
521     card_id

```

Output

#	Time	Action	Message	Duration / Fetch
21	10:22:39	CREATE TABLE credit_card_status AS -- Una CTE (Common Table Expression) es una subconsulta ...	Error Code: 1054. Unknown column 'credit_card_id' in 'field list'	0.000 sec
22	10:23:11	CREATE TABLE credit_card_status AS -- Una CTE (Common Table Expression) es una subconsulta ...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.500 sec

Anomenem la taula `credit_card_status` i fem servir les funcions de **FINESTRA** de **mySQL**.

Dins d'aquesta taula creem una subconsulta que podem invocar més tard amb el nom de `últimas`. És una CTE (common table expression) que s'inicia amb un `WITH` i amb un alias, per poder-la invocar després (aquí anomenada `ORDEN`).

Aquesta CTE contindrà una **FINESTRA** de **mySQL**. Concepte nou per a mí. El `ROW_NUMBER` genera una numeració per cada tarjeta (el `PARTITION` que ve després de `OVER`).
S'ordena per data (aquesta s'ha de modificar per fer servir el format `DATE` de any-mes-dia).

A continuació hem de fer els filtres oportuns. Fem servir `CASE` al que anomenem *estado* que funciona com un `if/else` en altres llenguatges. En concret ens porta un contador de control per sumar les operacions que son *declined*.

```

522 -- CASE ... END es una expresión condicional que evalúa una condición y devuelve un valor u otro
523 -- Aquí hay dos CASE anidados.
524 CASE
525 -- SUM(...) Si es una función agregada real: suma un valor por cada fila del grupo actual (ver GROUP BY).
526 WHEN SUM(
527 -- segundo CASE convierte la condición "no declinada" en numero para sumar. Por cada fila del grupo:
528 -- declined = 0 defino declined como aceptada cuando es 1 -> aporta 1 al SUM, hace de contador
529 -- declined = 1 (declinada) -> aporta 0 al SUM
530 CASE WHEN declined = 0 THEN 1 ELSE 0 END
531
532 -- Si la suma total de "aceptadas" dentro del grupo es 0,
533 -- significa que NINGUNA de las filas del grupo fue aceptada
534 ) = 0
535 THEN 'inactivo' -- las 3 últimas declinadas -> inactivo
536 ELSE 'activo' -- al menos 1 no declinada -> activo
537 END AS estado
538 FROM ultimas -- la CTE antes creada
539 -- filtro que limita el análisis a las 3 últimas.
540 WHERE orden <= 3
541 -- agrupa las filas restantes (máximo 3 por tarjeta) para
542 -- que el CASE/SUM del SELECT se calcule una vez por cada tarjeta,
543 GROUP BY card_id;
544

```

Output:

#	Time	Action	Message	Duration / Fetch
20	10:22:32	DROP TABLE 'transactions'.'credit_card_status'	0 row(s) affected	0.015 sec
21	10:22:39	CREATE TABLE credit_card_status AS -- Una CTE (Common Table Expression) es una subconsulta ...	Error Code: 1054. Unknown column 'credit_card_id' in 'field list'	0.000 sec
22	10:23:11	CREATE TABLE credit_card_status AS -- Una CTE (Common Table Expression) es una subconsulta ...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.500 sec

Si les tres últimes son declinades, aleshores el seu estat és *inactivo*. La resta son actives i així es pot veure a la consulta amb 5000 targetes actives.

```

448 FROM credit_card_status;
449 -- Consulta adicional: cuántas tarjetas quedaron "activas".
450 -- Como el resultado ya está guardado en una tabla física
451 SELECT COUNT(*) AS tarjetas_activas
452 FROM credit_card_status
453 WHERE estado = 'activo';

```

Result Grid

tarjetas_activas
5000

Result 11 x

Output:

#	Time	Action	Message	Duration / Fetch
26	10:44:21	SELECT * FROM credit_card_status	5000 row(s) returned	0.000 sec / 0.000 sec
27	10:44:24	SELECT COUNT(*) AS tarjetas_activas FROM credit_card_status WHERE estado = 'activo'	1 row(s) returned	0.015 sec / 0.000 sec

Exercici 2

Creo la taula products i l'omple amb les dades del CSV. Té 100 registres.

```

401 CREATE TABLE products (
402   id INT PRIMARY KEY,
403   product_name VARCHAR(50),
404   price DECIMAL(5,2),
405   colour VARCHAR(10),
406   weight DECIMAL(2,1),
407   warehouse_id VARCHAR(5),
408   category VARCHAR(15),
409   brand VARCHAR(20),
410   cost DECIMAL(5,2),
411   launch_date DATE
412 );
413
414 LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__products.csv'
415 INTO TABLE products
416 FIELDS TERMINATED BY ','
417 OPTIONALLY ENCLOSED BY '"'
418 LINES TERMINATED BY '\n'
419 IGNORE 1 ROWS
420 (id, product_name, @price_raw, colour, weight, warehouse_id, category, brand, @cost_raw, launch_date)
421 SET
422   price = REPLACE(@price_raw, '$', ''),
423   cost = REPLACE(@cost_raw, '$', '');
424

```

Output

#	Time	Action	Message	Duration / Fetch
22	12:13:06	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__products.csv'...	Error Code: 1366. Incorrect decimal value: '\$161.11' for column 'price' at row 1	0.016 sec
23	12:13:52	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__products.csv'...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.015 sec

Les principals dificultats son que price i cost tenen el caracter \$ davant i jo les he definit com a DECIMAL. Cal fer una transformació amb SET i REPLACE el \$ per un no res (").

La taula *transaction* té diversos productes a cada fila. No es pot unir *transaction* y *products* amb una *foreign key* normal. Cal fer una taula intermitja amb un inner join.

```

448 UNION ALL SELECT 9
449 UNION ALL SELECT 10
450 ) AS numeros
451 ON CHAR_LENGTH(transaction.product_ids)
452   - CHAR_LENGTH(REPLACE(transaction.product_ids, ',', '')) >= numeros.numero - 1
453 WHERE transaction.product_ids IS NOT NULL
454 AND transaction.product_ids <> '';
455
456 select * from transaction_product;
457
458

```

Result Grid

transaction_id	product_id
0010CCE6-CF15-4F25-85CE-A15308EF4D68	91
0010CCE6-CF15-4F25-85CE-A15308EF4D68	3
0010CCE6-CF15-4F25-85CE-A15308EF4D68	52
0011008C-179A-4DE6-B132-C8CC08921CFC	64
0011D40F-1F79-4802-970A-43320FE37CFF	85
00126FB9-7710-4EBC-AA92-00D7A9A15A40	71
00130606-1ADD-460D-85E6-A0DAD11F1C3C	92
00130606-1ADD-460D-85E6-A0DAD11F1C3C	74
00130606-1ADD-460D-85E6-A0DAD11F1C3C	80
0013803B-206D-4C03-94B7-63A2676EB9B4	13
0013803B-206D-4C03-94B7-63A2676EB9B4	34

transaction_product 22 x

Output

#	Time	Action	Message	Duration / Fetch
57	13:13:59	Error loading schema content	Error Code: 1356 View 'transactions.vistamarketing' references invalid table(s) or column(s) or function(...	
58	13:14:30	select * from transaction_product	253391 row(s) returned	0.000 sec / 0.313 sec

En aquest exercici hem de fer servir les **FINESTRES** de **mySQL**.

```

395
396 • SELECT card_id, declined, -- creo ventana y con partition reinicio contador de orden en cada tarjeta.ROW NUMBER Numera las filas sobre (OVER) credit_card_id
397     ROW_NUMBER() OVER (
398         PARTITION BY card_id
399         ORDER BY date(timestamp) DESC
400     ) AS orden
401 FROM transaction;
402 -- =====

```

The screenshot shows a MySQL IDE interface. The top pane displays a SQL query that uses a window function to assign an order number to each row within a partition of card_id. The bottom pane shows the 'Result Grid' with columns 'card_id', 'declined', and 'orden'. The 'orden' column contains values from 1 to 12 for card_id 'Cc5-4857'. Below the grid is the 'Action Output' log, which shows two successful actions: a SELECT query returning 888 rows and another SELECT query returning 100,000 rows.

card_id	declined	orden
Cc5-4857	0	1
Cc5-4857	0	2
Cc5-4857	0	3
Cc5-4857	0	4
Cc5-4857	0	5
Cc5-4857	0	6
Cc5-4857	0	7
Cc5-4857	0	8
Cc5-4857	0	9
Cc5-4857	0	10
Cc5-4857	0	11
Cc5-4857	0	12

creo la finestra amb **ROW NUMBER** per numerar les files sobre (**OVER**) **card_id**. Cada **card-id** associada a una operació té un número d'ordre. Hi ha tantes **card-id** com **transaction** hi haurà.

Com que els productes de *transaction* son una llista de numeros separats per una coma, els hem de manipular per crear una taula intermitja que relacioni la transacció amb cada producte per separat.

```

405 • CREATE TABLE transaction_product AS
406 SELECT
407     -- hago un alias de la primera columna
408     transaction.Id AS transaction_id,
409     -- Columna 2 del resultado: un único product_id
410     TRIM(
411         -- SUBSTRING_INDEX(texto, ',', -1) devuelve todo lo que hay.
412         SUBSTRING_INDEX(
413             -- SUBSTRING_INDEX(texto, ',', N) devuelve todo el texto. corta el texto por las comas
414             SUBSTRING_INDEX(transaction.product_ids, ',', numeros.numero),
415             -- con el -1 nos deja solo con el ÚLTIMO elemento de ese corte
416             ',', -1
417         )
418     -- TRIM() quita espacios en blanco sueltos
419 ) AS product_id

```

The screenshot shows a MySQL IDE interface. The top pane displays a SQL query that creates a table 'transaction_product' by selecting transaction IDs and product IDs from the 'transaction' table. The bottom pane shows the 'Action Output' log, which shows two successful actions: dropping the table 'transaction_product' and creating it with the specified query. The log indicates that 253,391 rows were affected.

Fem servir *trim* per separar la cadena de text desde la coma.

La taula creada *transaction_id* té les següents columnes:

```

405 • CREATE TABLE transaction_product AS
406 SELECT
407     -- hago un alias de la primera columna
408     transaction.Id AS transaction_id,
409     -- Columna 2 del resultado: un único product_id
410     TRIM(
411         -- SUBSTRING_INDEX(texto, ',', -1) devuelve todo lo que hay.
412         SUBSTRING_INDEX(
413             -- SUBSTRING_INDEX(texto, ',', N) devuelve todo el texto. corta el texto por las comas
414             SUBSTRING_INDEX(transaction.product_ids, ',', numeros.numero),
415             -- con el -1 nos deja solo con el ÚLTIMO elemento de ese corte

```

Result Grid | Filter Rows: | Exports: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
transaction_id	varchar(40)	NO		NULL	
product_id	varchar(50)	YES		NULL	

A la següent par de codi creem joins successius sobre numbers.number. Aquesta columna només existeix durant la consulta. No s'escriu a cap taula.

```

422 FROM transaction
423 -- para poder repetir cada transacción tantas veces como productos tenga.
424 JOIN (
425     -- generamos temporalmente 10 filas con los números del 1 al 10 mediante UNION ALL (cada SELECT aporta una fila).
426     SELECT 1 AS numero
427     UNION ALL SELECT 2
428     UNION ALL SELECT 3
429     UNION ALL SELECT 4
430     UNION ALL SELECT 5
431     UNION ALL SELECT 6
432     UNION ALL SELECT 7
433     UNION ALL SELECT 8
434     UNION ALL SELECT 9
435     UNION ALL SELECT 10 -- max. 10, no hay mas en el CSV
436 ) AS numeros

```

Afegim un màxim de 10 files. No hi ha més de 10 productes a una transacció. Si n'hi haguessin, se n'haurien d'afegir més UNION ALL SELECT n.

A continuació contruïm la taula amb els productes i les seves id de transacció.

```

437 -- Esta condición ON decide CUÁNTAS filas de "numeros" se combinan con cada transacción, para no generar de más ni de menos.
438 ON
439     -- CHAR_LENGTH(product_ids) = longitud total del texto,
440     -- por ejemplo CHAR_LENGTH("12,45,3") = 7.
441     CHAR_LENGTH(transaction.product_ids)
442     -- REPLACE(product_ids, ',', '') quita todas las comas del texto
443     -- CHAR_LENGTH(REPLACE(transaction.product_ids, ',', ''))
444     -- una transacción con solo 1 producto (0 comas) solo se
445     -- combinará con numero=1, y no generará filas de más con
446     -- numero=2 o numero=3 (que no le corresponden).
447     >= numeros.numero - 1
448 -- Aquí descartamos transacciones sin product_ids, para no generar filas vacías
449 WHERE transaction.product_ids IS NOT NULL
450 AND transaction.product_ids <> '';
451

```

Ara ja podem saber per exemple quantes vegades s'ha venut cada producte:

```
516 -- creo una consulta para ver las veces que se ha vendido cada producto
517 • SELECT products.id, products.product_name, COUNT(*) AS veces_vendido
518 FROM transaction_product
519 INNER JOIN products ON transaction_product.product_id = products.id
520 GROUP BY products.id, products.product_name
521 ORDER BY veces_vendido DESC;
```

Result Grid

	id	product_name	veces_vendido
52	riverlands the duel	2654	
29	Tully maester Tarly	2635	
21	duel Direwolf	2609	
16	the duel warden	2608	
66	mustafar jinn	2601	

Result 8 x

Output

#	Time	Action	Message	Duration / Fetch
18	09:46:59	describe credit_card_status	2 row(s) returned	0.000 sec / 0.000 sec
19	09:51:28	SELECT products.id, products.product_name, COUNT(*) AS veces_vendido FROM transaction_prod...	100 row(s) returned	1.015 sec / 0.000 sec

Hi ha 100 files, indica que la consulta és correcta i tenim el sumatori de les vegades que s'ha venut el producte per nom de producte.